

A1 — Build Your First CRUD API — Full Notes

Complete notes covering everything built across Stages 0–6 of the Backend Track Week 2 assignment (FastAPI, Python lane).

1. Core Concepts Recap

What is an API?

A defined set of "doors" (endpoints) that let a client (browser, curl, Swagger UI) talk to a server, without needing to know the server's internal code. Analogy: a restaurant menu — the kitchen (server logic) does the work; the menu (API) is the fixed set of things you're allowed to ask for.

HTTP methods → CRUD mapping

CRUD	HTTP Method	Meaning
Create	POST	Add new data
Read	GET	Retrieve data
Update	PUT	Replace/modify existing data
Delete	DELETE	Remove data

Status codes actually used in this project

- 200 OK — successful read
 - 201 Created — successful creation
 - 204 No Content — successful deletion (no body returned)
 - 400 Bad Request — invalid/empty input
 - 404 Not Found — resource doesn't exist
-

2. FastAPI Fundamentals

Basic pattern

```
python
```

```

from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello World"}

```

- `app = FastAPI()` — one object holding your whole API
- `@app.get(...)` / `@app.post(...)` / `@app.put(...)` / `@app.delete(...)` — decorators that *register* a function to run when a matching request arrives (they don't call the function immediately)
- Returning a dict → FastAPI auto-converts it to JSON
- `/docs` — auto-generated Swagger UI, built by FastAPI reading your registered routes

Path parameters

```

python

@app.get("/tasks/{task_id}")
def get_task_by_id(task_id: int):
    ...

```

- `{task_id}` in the path = a variable slot in the URL
- Adding `: int` gives you, for free: automatic type conversion, automatic validation (rejects non-numbers with a clear error), and better Swagger docs

Query parameters

- Any parameter **not** written inside `{ }` in the path is automatically treated as a query parameter
- Example: `done: bool = None` in a function whose path is just `/tasks` → becomes `?done=true` in the URL
- Giving a parameter a default value (`= None`) makes it optional

Request body (Pydantic models)

```

python

from pydantic import BaseModel

class CreateTask(BaseModel):
    title: str

```

- Must inherit from `BaseModel` — a plain class with type hints does **not** work for parsing JSON bodies
 - FastAPI auto-parses incoming JSON into this object, validates types, and gives you `.title` access
 - Required field: `title: str` (no default) — client must send it
 - Optional field: `title: str | None = None` — client may omit it
-

3. Key Bugs Encountered & Fixes (genuinely useful to remember)

Bug: mixing dicts and Pydantic objects

Storing a Pydantic object directly (`tasks[id] = student`) breaks square-bracket access (`["name"]`) later, since Pydantic objects use dot access (`.name`), not bracket access. **Fix:** convert to a plain dict before storing: `student.model_dump()` (note: `.dict()` is the deprecated older name for the same thing).

Bug: "create" vs "update" — append vs modify in place

When updating, building a new dict and `.append()`-ing it creates a **duplicate id** rather than updating the original. **Fix:** find the actual object reference in the list (`found_task = task` inside the loop), then modify it directly (`found_task["title"] = ...`) — since this is the same object sitting inside the list, no `.append()` needed.

Bug: id generation using `len(list)`

`next_id = len(tasks) + 1` breaks once items are deleted — the list shrinks, causing duplicate ids to be reassigned. **Fix:** always use **max existing id + 1**, never list length — ids must never be reused, even after deletion, to avoid ambiguous references.

```
python
next_id = -1
if len(tasks) != 0:
    for task in tasks:
        next_id = max(task["id"], next_id)
    next_id += 1
else:
    next_id = 1
```

Bug: unconditional field overwrite during partial update (PUT)

```
python
curr_task["title"] = details.title    # WRONG if title wasn't sent - wipes it to
```

Fix: only update a field if it was actually provided:

```
python

if details.title is not None:
    curr_task["title"] = details.title
if details.done is not None:
    curr_task["done"] = details.done
```

Bug: returning error messages with default 200 status

```
python

return {"error": "Task not found"}    # still sends HTTP 200!
```

Fix: use `HTTPException` to actually set the status code:

```
python

raise HTTPException(status_code=404, detail=f"Task {task_id} not found")
```

Validation: catching empty/whitespace-only input

```
python

if not create_title.title.strip():
    raise HTTPException(status_code=400, detail="Title cannot be empty")
```

`.strip()` alone catches both `""` and `" "` (whitespace-only), since stripping either results in an empty string.

4. Final Working Endpoints (Stage 0-4)

Method	Path	Behavior
GET	/	Returns API name, version, endpoints list
GET	/health	Returns {"status": "ok"}
GET	/tasks	Returns full task list
GET	/tasks/{task_id}	Returns one task or 404
POST	/tasks	Creates task (201), validates non-empty title (400)
PUT	/tasks/{task_id}	Partial update (title and/or done), 404/400 handled
DELETE	/tasks/{task_id}	Deletes task, returns 204, 404 if unknown

5. Environment & Tooling Notes

Global vs venv install

- Started with per-project `venv` (isolated packages), later switched to a **global** FastAPI/Uvicorn install for simplicity (`pip install fastapi uvicorn` from anywhere) — both approaches work; global is simpler for a single learner project, venv is better practice once juggling multiple projects with different dependency versions.
- If `uvicorn` command isn't recognized directly, run via: `python -m uvicorn main:app -reload`

Common terminal fixes hit along the way

- `main:app` needs exactly **one colon** — `main::app` or `main :app` will fail
- On Windows PowerShell, `curl` is aliased to `Invoke-WebRequest` and mishandles quotes — use `curl.exe` explicitly, and for POST/PUT bodies with JSON, write the JSON to a file and reference it with `-d "@body.json"` to avoid quote-escaping issues entirely
- `git commit --amend -m "new message"` edits only the **most recent** commit; if already pushed, follow with `git push --force` (safe for a solo repo)

Git / GitHub workflow used

```
powershell
```

```
git init
git add .
git commit -m "Stage X: description"
git push
```

- `.gitignore` must include `venv/`, `__pycache__`, and any temp test files (e.g. `body.json`) — created directly in VS Code (not via `echo`) to avoid encoding issues that silently break Git's ability to read the ignore rules
 - Never commit `venv/` — it's machine-specific and huge; share dependencies via a `requirements.txt` instead (`pip freeze > requirements.txt`)
-

6. Stage 5 & 6 — Swagger UI + README

- Swagger UI (`/docs`) is auto-built by FastAPI reading your registered routes — test full CRUD cycle there via "Try it out" before screenshotting for the README
 - Required README contents: description, run instructions, endpoint table, one `curl -i` output, Swagger screenshot
 - Final repo requirement: ≥ 6 meaningful commits (one per stage), public GitHub repo
-

7. What's Next (per curriculum)

Week 3 (Assignment A2) moves this same API from in-memory storage to a real **SQLite database** — same endpoints and behavior, but data now survives a server restart. The API layer barely changes; only the storage layer underneath does.